CS 305ES Plantilla para la actividad de codificación del módulo 5

Instrucciones

Siguiendo las instrucciones de los lineamientos y rúbrica para la verificación de la suma de comprobación de la actividad de codificación del módulo 5, sustituya el texto entre corchetes por la información pertinente con sus propias palabras.

1. Algoritmo de cifrado

El algoritmo hash sugerido para esta aplicación es SHA-256. Este algoritmo está clasificado como un algoritmo de la familia de funciones hash SHA-2 y está disponible como un nombre estándar para la clase MessageDigest en Java. SHA-256 produce un valor hash de 256 bits utilizando los datos de entrada sin importar cuánta longitud tenga este último. Oracle reconoce SHA-256 como uno de los algoritmos estándar utilizables mediante MessageDigest (Oracle, 2026).

2. Justificación

SHA-256 es adecuada para este contexto porque proporciona resistencia a colisiones: al menos en el caso de que dos entradas distintas den el mismo valor hash. Para una función hash criptográfica, la resistencia a colisiones quiere decir que tener dos entradas distintas que den el mismo resultado es computacionalmente inviable. NIST establece que SHA-256 proporciona una resistencia a colisiones de 128 bits (National Institute of Standards and Technology [NIST], 2024).

Esta propiedad es de interés porque la compañía necesita que los clientes puedan comprobar que la clave pública bajada no ha cambiado. Si hay cambio de contenido, el valor hash obtenido por la función de hashing debería ser diferente para las dos entradas. El cliente puede comparar el checksum dado con el checksum que previamente ha calculado, siendo capaz de detectar modificados del archivo o de los datos. Para implementar la solución en Java, se utiliza MessageDigest.getInstance("SHA-256") seguido de digest() para obtener el resumen, así como de una función usada para realizar la conversión a hexadecimal, bytesToHex (Oracle, 2026).

3. Generar suma de comprobación

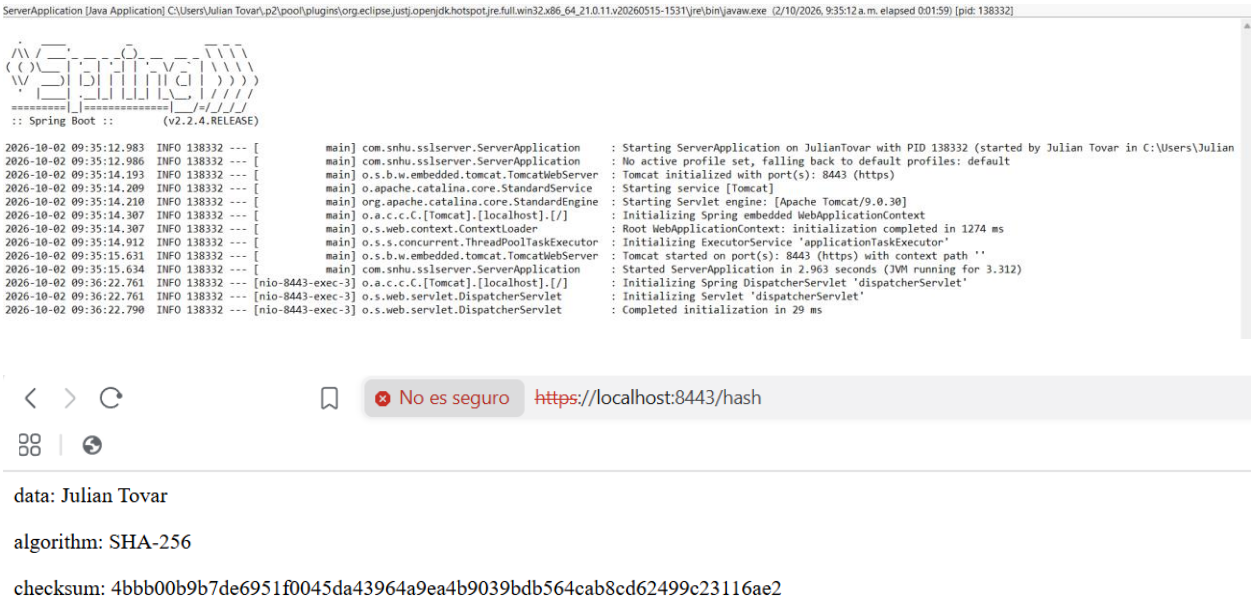
El código fue modificado para que utilizara la única cadena de datos que posee mi nombre + apellido. En primer lugar se genera el objeto MessageDigest con el algoritmo SHA-256. Seguidamente, se convierte la cadena a bytes mediante el Utf-8 y se extrae el valor hash mediante el método digest(). Por último, se convierte el resultado a su representación hexadecimal mediante el método bytesToHex.

El endpoint REST /hash devuelve al navegador la cadena de datos, el algoritmo y el valor hexadecimal de la suma de comprobación. En el presente proyecto se utiliza HTTPS mediante el puerto 8443 por lo que la información puede ser verificada en el navegador accediendo a la dirección https://localhost:8443/hash.

4. Verificación

Después de ejecutar la aplicación Spring Boot, se accede al endpoint /hash desde el navegador web seguro. La página muestra la cadena de datos utilizada, el algoritmo SHA-256 y el valor

hexadecimal generado. La captura de pantalla de esta página constituye la evidencia de que el programa generó correctamente la suma de comprobación para la cadena de datos única.



Referencias

- Oracle. (2026). *Java security standard algorithm names*. Oracle https://docs.oracle.com/en/java/javase/20/docs/specs/security/standard-names.html
- Oracle. (2026). *MessageDigest (Java SE 26 & JDK 26)*. Oracle. https://docs.oracle.com/en/java/javase/26/docs/api/java.base/java/security/MessageDigest.html
- National Institute of Standards and Technology. (2024). *Hash functions*. U.S. Department of Commerce. https://csrc.nist.gov/Projects/hash-functions